



FOM Hochschule für Oekonomie & Management
Hochschulzentrum Bonn

Berufsbegleitender Studiengang Wirtschaftsinformatik
zum
Bachelor of Science (B.Sc.)
6. Semester

Seminararbeit im Modul ERP-Systeme zum Thema

**End-to-End-Automatisierung von Produkt- und
Lagerprozessen in Odoo**

Dozent:	Dipl. Ing. Sandro Coletta
Name:	Lorenz Chylka, Leonard Reglin
Matrikelnummer:	736986, 740270
Studiengang:	Wirtschaftsinformatik
Semester:	6. Semester
Abgabedatum:	TT.MM.JJJJ
Wörter Hausarbeit:	5.391

II

Inhaltsverzeichnis

Abbildungsverzeichnis.....	IV
KI-Nutzung	V
Geschlechterinklusive Sprache	VI
Abkürzungsverzeichnis.....	VII
1 Einleitung	1
1.1 Problemstellung	2
1.2 Zielsetzung der Arbeit	3
2 Grundlagen.....	4
2.1 ERP-Systeme und Produktstammdaten	4
2.2 Barcodes und Produktidentifikation	4
2.3 Schnittstellen und Open-Food-Facts-API.....	4
3 Anforderungsanalyse	6
4 Odoo als Systembasis.....	7
4.1 Produkt- und Lagerverwaltung in Odoo	7
4.2 Beschaffungsprozess und Lieferantenzuordnung	10
4.3 Erweiterung durch ein Custom-Submodul	10
5 Konzeption und Umsetzung des Barcode-API-Submoduls	13
5.1 Prozessablauf vom Barcode-Scan bis zum Produktdatensatz	13
5.2 API-Abfrage und Datenübernahme aus open-food-facts	14
5.3 Integration in Lagerprozesse.....	15
6 Technische Analyse des Custom-Submoduls.....	17
6.1 Modulstruktur und Einordnung	17
6.2 Datenmodelle und Feldaufbau	18
6.3 ORM-Zugriffe und Persistenz	18
6.4 API-Aufruf, Validierung und Fehlerbehandlung	20
6.5 Transaktionslogik und Konsistenz.....	22
7 Fazit und Ausblick	23
Codeverzeichnis.....	24

III

Literaturverzeichnis	25
KI-Hilfsmittelverzeichnis	27

Abbildungsverzeichnis

1	Produktübersicht im Odoo-Prototyp.....	7
2	Verknüpfung eines Produkts mit einem Lieferanten	8
3	Erstellung einer Bestellung im Odoo-Prototyp.....	10
4	Bestätigung einer Bestellung	11
5	Validierung einer Bestellung.....	11
6	Abgeschlossene Bestellung im Beschaffungsprozess.....	12
7	Barcode-Importfenster im Prototyp	13
8	Automatisch angelegtes Produkt nach der API-Verarbeitung	14
9	Inventory-Übersicht mit Bestellfortschritt.....	15
10	Abgeschlossene Bestellung im Inventory-Kontext	16

KI-Nutzung

In dieser wissenschaftlichen Arbeit wurden zur sprachlichen Unterstützung bei der Erstellung des Textes KI-Modelle eingesetzt. Insbesondere wurden ChatGPT und DeepL zur Verbesserung der Grammatik, Struktur, Rechtschreibung und Lesbarkeit verwendet. Die inhaltliche Konzeption, Argumentation und jedwede Schlussfolgerungen liegen jedoch vollständig in der Verantwortung des Autors. Sollten Fremdwerke verwendet worden sein, sind diese entsprechend durch Fußnoten gekennzeichnet.

Geschlechterinklusive Sprache

In diesem Text wird aus Gründen der Lesbarkeit und Tradition häufig die maskuline Form verwendet. Dennoch sei betont, dass sich alle Ausführungen und Bezeichnungen geschlechtsübergreifend auf Personen aller Geschlechteridentitäten beziehen. Die Wahl der sprachlichen Form soll in keinem Fall als Ausschluss oder Diskriminierung anderer Geschlechter verstanden werden.

Abkürzungsverzeichnis

API	Application Programming Interface
EAN	European Article Number
ERP	Enterprise Resource Planning
IEC	International Electrotechnical Commission
ISO	International Organization for Standardization
IT	Informationstechnologie
KI	Künstliche Intelligenz
REST	Representational State Transfer

1 Einleitung

Die fortschreitende Digitalisierung von betrieblichen Wertschöpfungsprozessen führt dazu, dass Unternehmen in zunehmendem Maße auf integrierte ERP-Systeme (Enterprise Resource Planning) angewiesen sind. Diese werden genutzt, um Datenkonsistenz, Prozessqualität und operative Effizienz sicherzustellen. Besonders im Einzelhandel und in der Lebensmittelbranche besteht ein hoher Bedarf an aktuellen, strukturierten und fehlerfreien Produktstammdaten, da diese unmittelbar Grundlage für logistische, kaufmännische und kundenbezogene Prozesse und Entscheidungen sind.¹

Für kleine und mittlere Unternehmen stellt die manuelle Pflege dieser Stammdaten jedoch eine erhebliche Belastung dar. Uneinheitliche Datenquellen, redundante Datenerfassungen und fehlende Standardisierung führen zu inkonsistenten Datensätzen, Medienbrüchen und betrieblichen Ineffizienzen.² Da fehlerhafte oder unvollständige Stammdaten unmittelbare Auswirkungen auf Lagerprozesse, Verkaufsabläufe und betriebliche Entscheidungen haben, gilt das Stammdatenmanagement als ein kritischer Erfolgsfaktor für ERP-basierte Prozessautomatisierung. Hochwertige Daten bilden die Voraussetzung dafür, dass ERP-Systeme wie Odoo zuverlässig arbeiten und automatisierte Prozesse korrekt ausgeführt werden können.³

ERP-Systeme ermöglichen dabei die integrierte Abbildung zentraler Unternehmensprozesse und stellen eine einheitliche Datenbasis für verschiedene Funktionsbereiche bereit. Durch die zentrale Speicherung und Verarbeitung von Informationen können Redundanzen reduziert und Transparenz innerhalb der Organisation erhöht werden.⁴ Insbesondere im Bereich des Bestands- und Lagermanagements, sowie im Stammdatenmanagement tragen ERP-Systeme dazu bei, die Genauigkeit von Inventardaten zu verbessern und operative Abläufe effizienter zu gestalten.⁵

Darüber hinaus ermöglichen ERP-Systeme eine Echtzeitverarbeitung von Daten und fördern die bereichsübergreifende Zusammenarbeit innerhalb der Supply Chain. Studien zeigen, dass der Einsatz von ERP-Systemen signifikant zur Verbesserung der operativen Effizienz beitragen kann, da Prozesse besser koordiniert und Informationsflüsse optimiert werden können.⁶ Insbesondere durch die Automatisierung wiederkehrender Aufgaben und die Integration verschiedener Unternehmensbereiche können Fehlerquellen reduziert und

¹Vgl. Krcmar 2022.

²Vgl. Gronau 2017.

³Vgl. Lausen 2020.

⁴Vgl. Davenport 1998.

⁵Vgl. Romney und Steinbart 2018.

⁶Vgl. Chopra und Meindl 2016.

Entscheidungsprozesse beschleunigt werden.⁷

In der Lagerlogistik spielt die Qualität der Produktstammdaten eine zentrale Rolle. Fehlende oder inkonsistente Daten führen häufig zu falschen Bestandsbuchungen, ineffizienten Auftragsabwicklungen und erhöhten Kosten entlang der gesamten Lieferkette. Moderne ERP-Systeme ermöglichen eine präzisere Steuerung von Beständen und eine bessere Nachverfolgbarkeit von Warenströmen.⁸

Gleichzeitig steigt die Bedeutung externer Datenquellen für die Anreicherung von Produktinformationen. Offene Datenbanken wie open-food-facts stellen strukturierte und maschinenlesbare Produktdaten bereit, die über standardisierte Schnittstellen, sogenannte API's (Application Programming Interface), abgerufen werden können. Diese Entwicklung eröffnet neue Möglichkeiten zur Automatisierung der Stammdatenpflege und zur Reduktion manueller Eingaben, wodurch sowohl die Datenqualität als auch die Prozesseffizienz nachhaltig verbessert werden können.⁹

1.1 Problemstellung

Im Lebensmitteleinzelhandel müssen Produktstammdaten wie Produktbezeichnungen, Herstellerinformationen, Zutatenlisten, Allergenkennzeichnungen, Nährwerttabellen sowie EAN- und GTIN-Barcodes (EAN: European Article Number; GTIN: Global Trade Item Number) kontinuierlich aktualisiert werden, da sie die Grundlage für zahlreiche betriebliche Prozesse bilden. Die manuelle Erfassung dieser Informationen ist jedoch zeitintensiv, fehleranfällig und bindet erhebliche personelle Ressourcen. In der Praxis führt dies häufig zu inkonsistenten oder unvollständigen Datensätzen, redundanten Datenbeständen in unterschiedlichen Systemen sowie zu Übertragungsfehlern bei der Pflege und Aktualisierung der Informationen.

Wissenschaftliche Untersuchungen zeigen, dass eine unzureichende Datenqualität in ERP-Systemen maßgeblich zu ineffizienten Geschäftsprozessen, Fehlentscheidungen und erhöhten Prozesskosten beiträgt, was die steigende Relevanz eines strukturierten und qualitativ hochwertigen Stammdatenmanagements unterstreicht.¹⁰ Gleichzeitig bieten offene Produktdatenbanken wie open-food-facts standardisierte, strukturierte und maschinenlesbare Produktinformationen, die einen geeigneten Ansatzpunkt für die Automatisierung der Stammdatenpflege darstellen und dadurch das Potenzial besitzen, die Qualität und Aktualität von Produktdaten erheblich zu verbessern.¹¹

⁷Vgl. Chang 2014.

⁸Vgl. Chopra und Meindl 2016.

⁹Vgl. Open Food Facts 2026b.

¹⁰Vgl. Gronau 2017.

¹¹Vgl. Niedermeier 2026.

1.2 Zielsetzung der Arbeit

Ziel der vorliegenden Arbeit ist die Konzeption und prototypische Umsetzung einer End-to-End-Systemarchitektur, die eine automatisierte Produkt- und Lagerdatenverarbeitung in Odoo ermöglicht. Der entwickelte Prototyp soll demonstrieren, wie Barcode-Erfassung, API-basierte Datenanreicherung und darauf aufbauende ERP-Prozesse technisch miteinander verknüpft werden können, um die Qualität von Produktstammdaten zu verbessern.

Im Mittelpunkt steht dabei die Erfassung von Barcodes über geeignete Eingabegeräte, der anschließende Abruf und Vervollständigung der Produktdaten über die Open-Food-Facts-API sowie die darauf basierende Erstellung oder Aktualisierung eines Produktstammsatzes innerhalb von Odoo. Darüber hinaus soll gezeigt werden, wie die angereicherten Daten in die Lagerprozesse des Inventory-Moduls integriert werden können und wie sie nachgelagerte Verkaufsprozesse im Sales-Modul unterstützen.

Die Arbeit untersucht insgesamt, in welchem Ausmaß der Einsatz externer Datenquellen und teilautomatisierter Informationsflüsse dazu beiträgt, die Qualität des Stammdatenbestands zu erhöhen und zugleich die Prozesskosten in den Bereichen Produkthanlage, Lagerverwaltung und Verkauf nachhaltig zu reduzieren.

2 Grundlagen

Dieses Kapitel stellt zentrale Fachbegriffe und technologische Grundlagen bereit, die für die spätere Analyse und Implementierung erforderlich sind.

2.1 ERP-Systeme und Produktstammdaten

ERP-Systeme ermöglichen die integrierte Abbildung von Geschäftsprozessen und die zentrale Verwaltung betrieblicher Daten. Odoo ist ein modulares Open-Source-ERP-System mit integrierter PostgreSQL-Datenbank und über 30 Standardmodulen.

Relevante Module für die vorliegende Arbeit sind:

- **Inventory (Lagerverwaltung)** - Verwaltung von Beständen und Lagerbewegungen.
- **Sales (Vertrieb)** - Auftragsabwicklung und Versandprozesse.
- **Purchase (Einkauf)** - Bestell- und Beschaffungsprozesse.
- **Barcode-Modul** - Eigen erstelltes Modul zur Verarbeitung gescannter Barcodes und Aktionen in Odoo.¹²

2.2 Barcodes und Produktidentifikation

Barcodes, beispielsweise EAN-13, sind standardisierte maschinenlesbare Produktkennzeichen. In der Lagerlogistik verbessern sie die Identifikation von Artikeln, die Beschleunigung von Wareneingang und Inventur sowie die Reduktion manueller Fehler.¹³

2.3 Schnittstellen und Open-Food-Facts-API

open-food-facts ist eine offene, kollaborative Lebensmitteldatenbank, in der Produktinformationen wie Zutaten, Allergene, Nährwertangaben und weitere Angaben von Produktverpackungen erfasst werden. Die Datenbank wird als Open Data bereitgestellt und kann nach Angaben der Plattform von Dritten weiterverwendet werden.¹⁴ Für technische Anwendungen stellt open-food-facts eine API bereit, über die produktbezogene Informationen automatisiert abgerufen werden können.

Quellcode 1: API-Endpunkt für Produktabfragen über Barcode

¹²Vgl. Odoo S.A. 2026d.

¹³Vgl. ISO/IEC 15420 2026.

¹⁴Open Food Facts 2026a.

```
1 https://world.openfoodfacts.org/api/v0/product/{barcode}.json
```

Die API ermöglicht insbesondere den Zugriff auf Angaben wie Zutaten und Nährwerte und kann dadurch zur automatisierten Anreicherung von Produktstammdaten in ERP-Systemen genutzt werden.

3 Anforderungsanalyse

An das Submodul werden sowohl funktionale als auch nichtfunktionale Anforderungen gestellt. Im Mittelpunkt steht zunächst die Erfassung eines Barcodes über ein geeignetes Endgerät. Dieser Barcode dient als eindeutiger Identifikator für das jeweilige Produkt und bildet die Grundlage für die anschließende Datenabfrage. Über die Open-Food-Facts-API werden die zugehörigen Produktinformationen abgerufen, sofern der Barcode in der Datenbank vorhanden ist. Die empfangenen Produktdaten sollen anschließend automatisiert verarbeitet und in Odoo übernommen werden.

Innerhalb des ERP-Systems muss das Submodul in der Lage sein, ein neues Produkt automatisch anzulegen oder ein bereits vorhandenes Produkt zu aktualisieren. Dadurch wird sichergestellt, dass die Informationen eines Produktes nicht manuell gepflegt werden müssen und der Datenbestand im System aktuell bleibt. Zusätzlich ist eine Zuordnung des Artikels zum Lagerbestand erforderlich, damit das Produkt nicht nur als Stammdatensatz existiert, sondern auch in den operativen Lagerprozessen verwendet werden kann. Damit wird eine Verbindung zwischen Produktidentifikation, Stammdatenverwaltung und Bestandsführung hergestellt.

Darüber hinaus muss das Submodul den Verkaufsprozess im Sales-Modul unterstützen. Die erfassten und gepflegten Produktdaten sollen daher so bereitgestellt werden, dass sie für Verkaufsangebote, Aufträge und weitere vertriebsbezogene Prozesse nutzbar sind. Das Submodul übernimmt damit eine Schnittstellenfunktion zwischen externer Produktdatenquelle, interner Warenwirtschaft und Verkaufsabwicklung.

Es sind auch mehrere nichtfunktionale Anforderungen zu berücksichtigen. Die Verarbeitung der API-Antworten muss performant erfolgen, damit Barcode-Erfassung und Produkthanlage ohne spürbare Verzögerungen durchgeführt werden können. Gleichzeitig ist Fehlertoleranz erforderlich, insbesondere bei Barcodes, die in der Open-Food-Facts-Datenbank nicht bekannt sind oder bei denen unvollständige Produktdaten zurückgegeben werden. In solchen Fällen muss das System kontrolliert reagieren und darf keine inkonsistenten Datensätze erzeugen.

Zudem soll das Submodul erweiterbar gestaltet werden, sodass künftig weitere Datenquellen ergänzt werden können. Eine feste Bindung an eine einzelne externe API wäre daher zu vermeiden. Abschließend ist eine konsistente Datenhaltung im ERP-System sicherzustellen. Produktdaten, Lagerinformationen und vertriebsrelevante Angaben müssen eindeutig, widerspruchsfrei und nachvollziehbar gespeichert werden, damit das Submodul zuverlässig in die bestehenden Odoo-Prozesse integriert werden kann.

4 Odoo als Systembasis

Odoo bildet die technische Systembasis des entwickelten Prototyps, da zentrale Produkt-, Lager- und Verkaufsprozesse innerhalb einer integrierten ERP-Umgebung abgebildet werden können. Das System ist modular aufgebaut und stellt unterschiedliche Anwendungen bereit, die auf einer gemeinsamen Datenbasis miteinander verbunden werden. Für die vorliegende Arbeit sind insbesondere die Produktverwaltung, die Lagerverwaltung sowie die technische Erweiterbarkeit durch individuell entwickelte Module relevant.

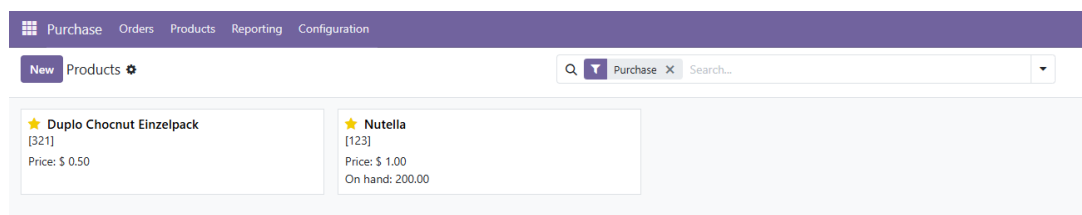
Die im Prototyp verwendete Barcode-Funktionalität basiert nicht auf dem von Odoo standardmäßig bereitgestellten Barcode-Modul. Stattdessen wird ein eigenständig programmiertes Custom-Submodul eingesetzt, das Barcodes entgegennimmt, externe Produktdaten über die Open-Food-Facts-API abrufen und diese Daten anschließend in die Produkt- und Lagerstrukturen von Odoo überführt. Odoo wird damit nicht als vollständige Standardlösung verwendet, sondern als ERP-Systembasis, die durch eine projektspezifische Erweiterung an den vorliegenden Anwendungsfall angepasst wird.

4.1 Produkt- und Lagerverwaltung in Odoo

Die Produktverwaltung in Odoo erfolgt über zentrale Produktmodelle, in denen grundlegende Produktstammdaten wie Produktbezeichnung, Verkaufspreis, Kategorie und Barcode gespeichert werden können. Dabei wird zwischen Produktvorlagen und konkreten Produktvarianten unterschieden. Produktvorlagen dienen der Verwaltung allgemeiner Produktinformationen, während Produktvarianten spezifische Ausprägungen eines Produkts abbilden können.¹⁵

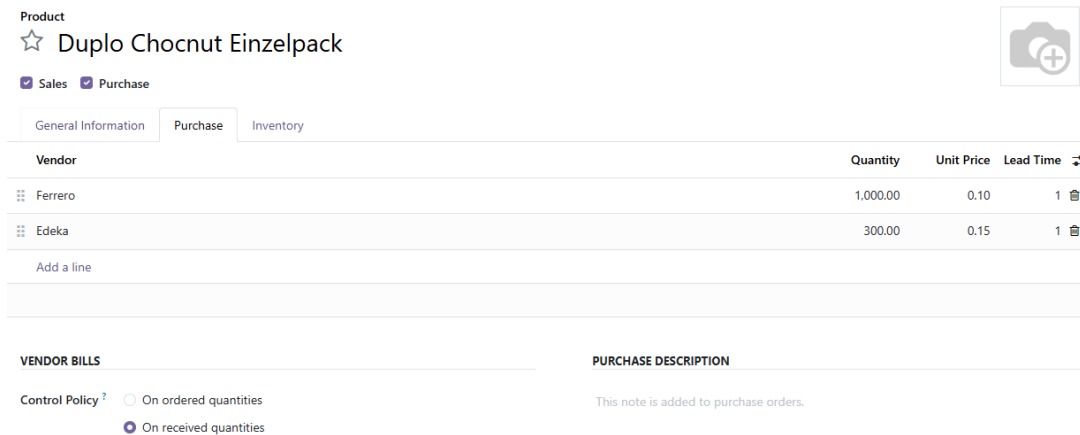
Abbildung 1 zeigt die Produktübersicht im Prototyp, während Abbildung 2 die Verknüpfung eines Produkts mit einem Lieferanten dokumentiert. Beide Ansichten sind für die spätere automatische Produkthanlage und die Beschaffungslogik relevant.

Abbildung 1: Produktübersicht im Odoo-Prototyp



Quelle: Eigene Darstellung

¹⁵Odoo S.A. 2026c.

Abbildung 2: Verknüpfung eines Produkts mit einem Lieferanten


The screenshot shows the Odoo Purchase Order form for the product 'Duplo Chocnut Einzelpack'. The form is divided into several sections:

- Product:** Duplo Chocnut Einzelpack (with a star icon and a camera icon).
- Buttons:** Sales (disabled), Purchase (active).
- Tabs:** General Information, Purchase (selected), Inventory.
- Table:** A table with columns: Vendor, Quantity, Unit Price, Lead Time, and an icon column. It lists two vendors: Ferrero (Quantity: 1,000.00, Unit Price: 0.10, Lead Time: 1) and Edeka (Quantity: 300.00, Unit Price: 0.15, Lead Time: 1). There is an 'Add a line' button below the table.
- VENDOR BILLS:** Control Policy? with radio buttons for 'On ordered quantities' (selected) and 'On received quantities'.
- PURCHASE DESCRIPTION:** This note is added to purchase orders.

Quelle: Eigene Darstellung

Für die Lagerverwaltung stellt Odoo das Inventory-Modul bereit. Dieses ermöglicht die Verwaltung von Lagerorten, Beständen, Wareneingängen, internen Umlagerungen und Warenausgängen. Lagerbewegungen werden systemseitig dokumentiert und mit den entsprechenden Produkten verknüpft. Dadurch können Bestände nicht isoliert, sondern im Zusammenhang mit den zugrunde liegenden Produktstammdaten verarbeitet werden.¹⁶ Im Kontext der vorliegenden Arbeit ist diese Verbindung zwischen Produkt- und Lagerverwaltung wesentlich. Ein Produkt, das durch das Custom-Submodul automatisch angelegt oder aktualisiert wird, kann anschließend unmittelbar in lagerbezogene Prozesse eingebunden werden. Dazu zählen beispielsweise die Zuordnung zu einem Lagerbestand, die spätere Verbuchung von Wareneingängen oder die Verwendung in Verkaufsprozessen. Die Produktdaten dienen somit als Stammdatenbasis für nachgelagerte ERP-Prozesse.

Die Verwendung von Barcodes unterstützt dabei die eindeutige Identifikation von Produkten. In Odoo können Barcodes als Bestandteil der Produktstammdaten gespeichert werden. Dadurch kann ein gescannter Code einem bestehenden Produktdatensatz zugeordnet werden. Für den entwickelten Prototyp wird diese Möglichkeit genutzt, jedoch nicht über das standardmäßige Odoo-Barcode-Modul umgesetzt. Der Barcode wird vielmehr durch ein eigenes Submodul verarbeitet, das die Produktidentifikation mit einer externen Datenabfrage verbindet.¹⁷

Zur Umsetzung der automatisierten Produkthanlage wird Odoo durch ein eigenständig entwickeltes Custom-Submodul erweitert. Dieses Submodul ist nicht mit dem von Odoo bereitgestellten Barcode-Modul gleichzusetzen. Es handelt sich um eine projektspezifische

¹⁶Odoo S.A. 2026a.

¹⁷Odoo S.A. 2026c.

Erweiterung, die ausschließlich für den in dieser Arbeit betrachteten Prozess entwickelt wird. Die Aufgabe des Submoduls besteht darin, einen eingegebenen oder gescannten Barcode zu verarbeiten, eine Anfrage an die Open-Food-Facts-API zu stellen und die zurückgegebenen Produktinformationen in Odoo weiterzuverarbeiten.

Die technische Erweiterbarkeit von Odoo wird durch das modulare Entwicklungsmodell ermöglicht. Ein Odoo-Modul besteht typischerweise aus einer Manifest-Datei, Python-Dateien für die Geschäftslogik sowie XML-Dateien zur Definition von Ansichten, Menüs und weiteren Konfigurationen.¹⁸ Die Geschäftslogik wird über Python-Klassen umgesetzt, die auf das Object-Relational-Mapping von Odoo zugreifen. Dadurch können bestehende Datenmodelle erweitert oder neue Modelle definiert werden.¹⁹

Das entwickelte Custom-Submodul wird zwischen Barcode-Erfassung, externer API-Abfrage und Produktverwaltung positioniert. Nach Eingabe eines Barcodes wird zunächst geprüft, ob in Odoo bereits ein Produkt mit diesem Barcode existiert. Falls ein entsprechender Datensatz vorhanden ist, können die vorhandenen Informationen aktualisiert oder ergänzt werden. Falls kein Produkt gefunden wird, wird ein neuer Produktstammsatz angelegt. Die über die Open-Food-Facts-API abgerufenen Daten werden dabei auf die relevanten Felder des Odoo-Produktmodells abgebildet. Hierzu zählen insbesondere Produktname, Marke, Barcode und weitere verfügbare Produktinformationen.

Durch die Umsetzung als eigenständiges Submodul bleibt die Standardfunktionalität von Odoo unverändert. Dieses Vorgehen reduziert Abhängigkeiten vom Kernsystem und erleichtert spätere Anpassungen. Gleichzeitig kann die Lösung erweitert werden, etwa durch zusätzliche Datenquellen, weitere Validierungsregeln oder eine detailliertere Zuordnung von Produktinformationen zu Lager- und Verkaufsprozessen. Das Custom-Submodul übernimmt damit die Rolle einer Integrationskomponente zwischen externer Produktdatenquelle und ERP-interner Stammdatenverwaltung.

1. Barcode-Eingabe.
2. API-Request zu open-food-facts.
3. Mapping der Daten auf Odoo-Modelle.
4. Speicherung von Produkt und Lagerbestand.
5. Optionale Übergabe an Verkaufsprozesse.

¹⁸Odoo S.A. 2026b.

¹⁹Odoo S.A. 2026b.

4.2 Beschaffungsprozess und Lieferantenzuordnung

Die prototypische Umsetzung berücksichtigt neben der reinen Produkthanlage auch die Zuordnung zu Lieferanten und den nachgelagerten Einkaufsprozess in Odoo. Die folgenden Abbildungen zeigen exemplarisch, wie eine Bestellung erzeugt, bestätigt, validiert und anschließend als abgeschlossen dokumentiert wird.

Abbildung 3: Erstellung einer Bestellung im Odoo-Prototyp

New Requests for Quotation P00002

Send RFQ Confirm Order Print Cancel

RFQ RFQ Sent Purchase Order

Request for Quotation
☆ P00002

Vendor ? Edeka

Order Deadline ? May 9, 11:38 AM

Vendor Reference ?

Expected Arrival ? May 10, 11:38 AM No data yet

☐ Ask confirmation

Deliver To ? My Company: Receipts

Products Other Information

Product	Quantity	Unit Price	Taxes	Amount
[321] Duplo Chocnut Einzelpack	300.00	0.15	15%	\$ 45.00

Add a product Add a section Add a note Catalog

Define your terms and conditions ...

Untaxed Amount: \$ 45.00
Tax 15%: \$ 6.75
Total: \$ 51.75

Quelle: Eigene Darstellung

4.3 Erweiterung durch ein Custom-Submodul

Für die prototypische Implementierung werden Python 3.12, Odoo 19 On-Premise, PostgreSQL ab Version 12 sowie ein Odoo-Custom-Addon für die API-Integration verwendet. Python dient dabei als Programmiersprache für die Geschäftslogik des Submoduls. Odoo stellt die ERP-Systemumgebung bereit, während PostgreSQL als relationale Datenbank für die Speicherung der ERP-Daten genutzt wird. Das Custom-Addon übernimmt die technische Verbindung zwischen Barcode-Eingabe, API-Abfrage und Produktdatenverarbeitung innerhalb von Odoo.

Die Erweiterung wird so konzipiert, dass die API-Integration nicht direkt im Odoo-Kernsystem umgesetzt wird, sondern in einer separaten Modullogik. Dadurch bleibt die Wartbarkeit der Lösung erhalten. Änderungen an der API-Verarbeitung, an Mapping-Regeln oder an Validierungsmechanismen können innerhalb des Custom-Submoduls vorgenommen werden, ohne die Standardmodule von Odoo unmittelbar zu verändern.

Abbildung 4: Bestätigung einer Bestellung

New Requests for Quotation P00002

Receipt 1

Receive Upload Bill Send PO Acknowledge Print Cancel RFQ RFQ Sent **Purchase Order**

Purchase Order
☆ **P00002**

Vendor [?] Edeka Confirmation Date May 9, 11:39 AM
Vendor Reference [?] Expected Arrival [?] May 10, 11:38 AM
☐ Ask confirmation
Deliver To [?] My Company: Receipts

Products Other Information

Product	Quantity	Received	Billed	Unit Price	Taxes	Amount
[321] Duplo Chocnut Einzelpack	300.00	0.00	0.00	0.15	15%	\$ 45.00
Add a product Add a section Add a note Catalog						

Define your terms and conditions ...

Untaxed Amount: **\$ 45.00**
Tax 15%: \$ 6.75
Total: **\$ 51.75**

Quelle: Eigene Darstellung

Abbildung 5: Validierung einer Bestellung

New Requests for Quotation / P00002 WH/IN/00002

Moves

Validate Print Return Cancel Draft **Ready** Done

☆ **WH/IN/00002**

Receive From Edeka → Scheduled Date [?] May 10, 11:38 AM
Operation Type My Company: Receipts Deadline [?] May 10, 11:38 AM
Source Document [?] P00002

Operations Additional Info Note

Product	Demand	Quantity
[321] Duplo Chocnut Einzelpack	300.00	300.00
Add a Product		

Quelle: Eigene Darstellung

Abbildung 6: Abgeschlossene Bestellung im Beschaffungsprozess

New

Requests for Quotation / P00002
WH/IN/00002

Moves

Print

Return

Draft

Ready

Done

☆ WH/IN/00002

Receive FromEdeka

Scheduled Date[?]May 10, 11:38 AM

Operation TypeMy Company: Receipts

Effective Date[?]May 9, 11:41 AM

Source Document[?]P00002

Operations

Additional Info

Note

Product	Demand	Quantity	
[321] Duplo Chocnut Einzelpack	300.00	300.00	

Quelle: Eigene Darstellung

5 Konzeption und Umsetzung des Barcode-API-Submoduls

Dieses Kapitel beschreibt die Konzeption und technische Umsetzung des Submoduls. Der Schwerpunkt liegt auf dem Prozessablauf vom Barcode-Scan bis zum Produktdatensatz, der API-Integration und der Einbindung der erzeugten oder aktualisierten Produktdaten in Odoo.

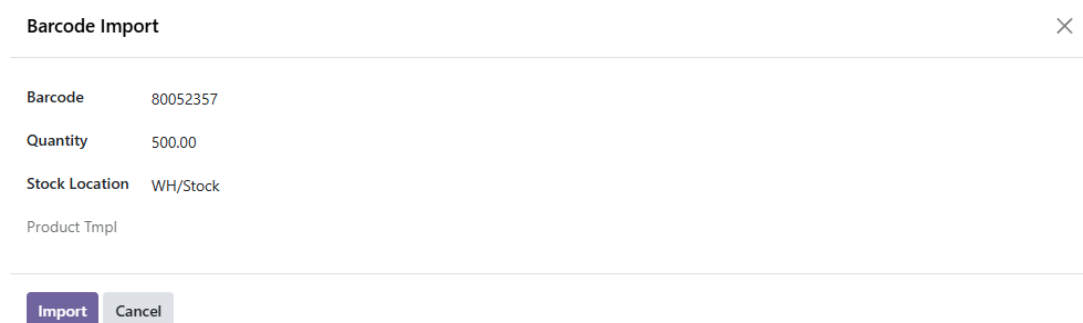
5.1 Prozessablauf vom Barcode-Scan bis zum Produktdatensatz

Der Prozess beginnt mit der Erfassung eines Barcodes. Diese Erfassung kann über ein Barcode-fähiges Endgerät oder durch manuelle Eingabe erfolgen. Der Barcode wird anschließend vom Custom-Submodul entgegengenommen und als eindeutiges Identifikationsmerkmal für die weitere Verarbeitung genutzt. Im ersten Verarbeitungsschritt wird geprüft, ob in Odoo bereits ein Produktdatensatz mit diesem Barcode vorhanden ist. Diese Prüfung verhindert, dass identische Produkte mehrfach angelegt werden.

Ist bereits ein Produkt mit dem erfassten Barcode vorhanden, können bestehende Informationen aktualisiert oder ergänzt werden. Ist kein entsprechender Datensatz vorhanden, wird auf Grundlage der extern abgerufenen Produktinformationen ein neuer Produktstamm-satz angelegt. Der Prozess umfasst somit das Empfangen des Barcodes, den Aufruf des Custom-Addons, die Abfrage der externen Datenquelle, die Verarbeitung der API-Antwort sowie das Erstellen oder Aktualisieren des Produktdatensatzes in Odoo.

Abbildung 7 zeigt die Erfassung des Barcodes im Importfenster, Abbildung 8 die automatische Anlage des Produktdatensatzes nach der externen Datenanreicherung.

Abbildung 7: Barcode-Importfenster im Prototyp



Barcode Import	
Barcode	80052357
Quantity	500.00
Stock Location	WH/Stock
Product Tmpl	

Import Cancel

Quelle: Eigene Darstellung

Abbildung 8: Automatisch angelegtes Produkt nach der API-Verarbeitung

The screenshot shows the Odoo Product form for 'Duplo Chocnut Einzelpack'. The interface includes a top navigation bar with 'Purchase', 'Orders', 'Products', 'Reporting', and 'Configuration'. Below this, there's a breadcrumb trail: 'New / [123] Nutella / [123] Nutella'. The product name 'Duplo Chocnut Einzelpack' is displayed with a star icon and a camera icon for uploading an image. The form is divided into tabs: 'General Information', 'Purchase', and 'Inventory'. The 'General Information' tab is active, showing fields for 'Product Type' (set to 'Goods'), 'Track Inventory' (unchecked), 'Sales Price' (\$ 0.50), 'Sales Taxes' (15%), 'Cost' (\$ 0.10), 'Purchase Taxes' (15%), 'Category' (Snacks), 'Reference', and 'Barcode' (80052357). A note at the bottom states: 'INTERNAL NOTES: This note is only for internal purposes.'

Quelle: Eigene Darstellung

5.2 API-Abfrage und Datenübernahme aus open-food-facts

Die produktbezogene Datenabfrage erfolgt über einen Python-Service, der anhand des Barcodes eine Anfrage an die Open-Food-Facts-API sendet. Die Anfrage nutzt die Barcode-basierte Produktressource der API. Der technische Aufruf kann vereinfacht wie folgt dargestellt werden:

Quellcode 2: Produktbezogene API-Abfrage

```
1 GET /api/v0/product/{barcode}.json
```

Nach Eingang der API-Antwort werden die relevanten Datenfelder extrahiert und auf die entsprechenden Odoo-Modelle übertragen. Dazu gehören insbesondere Produktname, Hersteller- beziehungsweise Markeninformationen, Barcode sowie verfügbare Nährwert- und Produktinformationen. Die Datenübernahme erfolgt nicht ungeprüft, sondern muss berücksichtigen, dass externe Datensätze unvollständig sein können. Daher ist eine Verarbeitung erforderlich, die fehlende Felder erkennt und nur geeignete Informationen in die Produktverwaltung überführt.²⁰

Die übernommene Datenstruktur muss mit den Datenmodellen von Odoo kompatibel sein. Aus diesem Grund werden die aus der API erhaltenen Informationen in ein internes Mapping überführt. Dieses Mapping legt fest, welche API-Felder welchen Odoo-Feldern ent-

²⁰Open Food Facts 2026a.

sprechen. Auf dieser Grundlage können Produktdaten erstellt, aktualisiert und anschließend in nachgelagerten Prozessen verwendet werden.

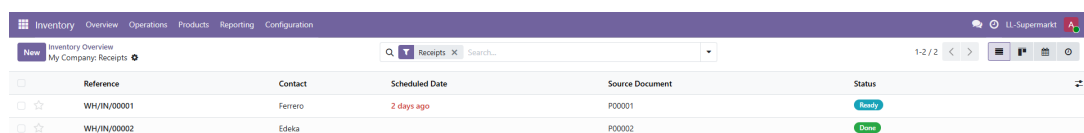
5.3 Integration in Lagerprozesse

Sobald ein Produkt in Odoo vorhanden ist, kann es im Inventory-Modul verwendet werden. Dadurch wird es möglich, das Produkt in Wareneingängen zu verbuchen, Lagerbewegungen zu dokumentieren, Bestände zu verwalten oder das Produkt im Rahmen von Kommissionierungsprozessen auszulagern. Die automatische Produkthanlage bildet damit die Grundlage für eine anschließende lagerbezogene Verarbeitung.

Die Integration in Lagerprozesse beruht darauf, dass Produktstammdaten und Lagerdaten im ERP-System miteinander verknüpft sind. Ein durch das Custom-Submodul angelegtes oder aktualisiertes Produkt kann dadurch denselben Prozessen zugeführt werden wie ein manuell angelegtes Produkt. Zusätzlich können die Produktdaten im Sales-Modul für Bestellungen und Verkaufsprozesse genutzt werden. Die Schnittstelle zwischen Barcode-Erfassung, externer Produktdatenquelle und Odoo-Produktmodell ermöglicht somit einen durchgängigen Prozess von der Produktidentifikation bis zur ERP-internen Weiterverarbeitung.

Die Abbildung 9 dokumentiert den Bestellfortschritt im Inventory-Kontext; Abbildung 10 zeigt den abgeschlossenen Zustand nach der Verarbeitung.

Abbildung 9: Inventory-Übersicht mit Bestellfortschritt



	Reference	Contact	Scheduled Date	Source Document	Status
☐ ☆	WH/IN/00001	Ferrero	2 days ago	P00001	Ready
☐ ☆	WH/IN/00002	Edeka		P00002	Done

Quelle: Eigene Darstellung

Abbildung 10: Abgeschlossene Bestellung im Inventory-Kontext

New

Requests for Quotation / P00002
WH/IN/00002

Moves

Print

Return

Draft

Ready

Done

☆ WH/IN/00002

Receive FromEdeka

Scheduled Date[?]May 10, 11:38 AM

Operation TypeMy Company: Receipts

Effective Date[?]May 9, 11:41 AM

Source Document[?]P00002

Operations

Additional Info

Note

Product	Demand	Quantity	
[321] Duplo Chocnut Einzelpack	300.00	300.00	

Quelle: Eigene Darstellung

6 Technische Analyse des Custom-Submoduls

6.1 Modulstruktur und Einordnung

Die vorliegende Umsetzung besteht nicht nur aus einem einzelnen API-Aufruf, sondern aus einem vollständigen Odoo-Addon mit klar getrennten Schichten. Die Modulstruktur im Verzeichnis `barcode_open_food_facts` kann in vier funktionale Bereiche unterteilt werden: Manifest, Modelle, Views und Sicherheit. Eine dedizierte Teststruktur ist in der aktuellen Repository-Version nicht vorhanden.

- `__manifest__.py`: Metadaten, Abhängigkeiten und Lade-Reihenfolge der XML- und CSV-Dateien.
- `models/`: Fachlogik für den Import-Workflow und die persistente Ablage der Zusatzdaten.
- `views/`: Formular, Actions und Menüs für den fachlichen Zugriff in der Odoo-Oberfläche.
- `security/ir.model.access.csv`: Zugriff für interne Benutzer auf Wizard und Produktmodell.

Das Modul ist als Anwendung installierbar und hängt technisch von `product`, `stock` und `web` ab. Damit wird die Produkthanlage direkt an den Odoo-Standard gekoppelt und gleichzeitig die Lagerbuchung vorbereitet.

Quellcode 3: Auszug aus `__manifest__.py`

```
1 {  
2     "name": "Barcode open-food-facts",  
3     "version": "19.0.1.0.0",  
4     "depends": ["product", "stock", "web"],  
5     "data": [  
6         "security/ir.model.access.csv",  
7         "views/barcode_import_wizard_views.xml",  
8         "views/barcode_import_actions.xml",  
9         "views/barcode_off_product_views.xml",  
10        "views/barcode_menus.xml",  
11    ],  
12    "installable": True,  
13    "application": True,  
14 }
```


6.2 Datenmodelle und Feldaufbau

Die Implementierung verwendet zwei Odoo-Modelle. `barcode.off.import.wizard` ist ein `TransientModel` für die Importmaske. `barcode.off.product` ist ein persistentes Modell zur Ablage der erweiterten Open-Food-Facts-Daten pro Produktvorlage. Damit trennt die Lösung den UI-nahen Importvorgang von der dauerhaften Datenspeicherung.

Quellcode 4: Persistentes Modell für Open-Food-Facts-Daten

```

1 class BarcodeOffProduct(models.Model):
2     _name = "barcode.off.product"
3     _description = "open-food-facts Product Data"
4
5     product_tmpl_id = fields.Many2one("product.template", required=True,
6         ondelete="cascade")
7     barcode = fields.Char(index=True)
8     brand = fields.Char(string="Brand")
9     quantity_str = fields.Char(string="Quantity (Text)")
10    image_url = fields.Char(string="Image URL (OFF)")
11    ingredients_text = fields.Text(string="Ingredients")
12    allergens = fields.Text(string="Allergens")
13    energy_kcal = fields.Float(string="Energy (kcal/100g)")
14    fat_g = fields.Float(string="Fat (g/100g)")
15    carbohydrates_g = fields.Float(string="Carbohydrates (g/100g)")
16    proteins_g = fields.Float(string="Proteins (g/100g)")
17    salt_g = fields.Float(string="Salt (g/100g)")
18    nutri_score = fields.Char(string="Nutri-Score")
19    ecoscore_grade = fields.Char(string="Eco-Score")
20    labels = fields.Char(string="Labels")
21    off_json_data = fields.Json(string="Raw OFF Data")

```

Die Feldstruktur zeigt bereits die fachliche Zielrichtung der Lösung. Neben Basisdaten wie Barcode, Marke und Menge werden auch Ernährungs- und Zutateninformationen sowie die Rohantwort der API abgelegt. Dadurch bleibt der Import auditierbar und spätere Nachverarbeitung ist möglich.

6.3 ORM-Zugriffe und Persistenz

Der Import-Workflow arbeitet vollständig über den Odoo-ORM-Zugriff. Es werden `env`-Zugriffe, `search`, `create`, `write`, `ensure_one` und `sudo()` eingesetzt. Die Entscheidung, ob ein Produkt neu angelegt oder aktualisiert wird, erfolgt nicht in SQL, sondern

in der Modelllogik des Wizards. Dadurch bleibt die Implementierung Odoo-konform und nutzt die Transaktionssteuerung des Frameworks.

Das zentrale Muster ist die Dublettenprüfung über den Barcode. Vor einer Neuanlage wird `product.product` nach dem Barcode durchsucht. Existiert bereits ein Datensatz, wird die zugehörige Produktvorlage aktualisiert. Andernfalls wird eine neue Vorlage mit Variante erzeugt und der Barcode auf der Variante gesetzt.

Quellcode 5: Dublettenprüfung und Upsert über den ORM

```

1 def _upsert_product(self, barcode, product_data):
2     product = self.env["product.product"].search([("barcode", "=", barcode)
3         ], limit=1)
4     name = self._prepare_product_name(product_data, barcode)
5     category = self._prepare_category(product_data)
6     description = self._prepare_description(product_data)
7
8     vals = {"name": name}
9     if category:
10         vals["categ_id"] = category.id
11     if description:
12         vals["description"] = description
13
14     if product:
15         product.product_tmpl_id.write(vals)
16         existing_product = product
17     else:
18         vals["type"] = "consu"
19         template = self.env["product.template"].create(vals)
20         template.product_variant_id.barcode = barcode
21         existing_product = template.product_variant_id
22
23     self._save_off_data(existing_product.product_tmpl_id, barcode,
24         product_data)
25     return existing_product

```

Die Persistenz der Zusatzdaten erfolgt anschließend in `barcode.off.product`. Die Methode legt einen vorhandenen Datensatz per `search` auf Basis der Produktvorlage zurück und schreibt ihn bei Bedarf mit `write` fort. Falls noch kein Datensatz existiert, wird er neu erstellt.

Quellcode 6: Mapping der API-Felder auf Odoo-Felder

```

1 off_vals = {
2     "product_tmpl_id": product_tmpl.id,
3     "barcode": barcode,
4     "brand": product_data.get("brands", ""),
5     "quantity_str": product_data.get("quantity", ""),
6     "image_url": product_data.get("image_url", ""),
7     "ingredients_text": product_data.get("ingredients_text", ""),
8     "allergens": product_data.get("allergens", ""),
9     "energy_kcal": product_data.get("energy_kcal_100g", 0.0),
10    "fat_g": product_data.get("fat_100g", 0.0),
11    "carbohydrates_g": product_data.get("carbohydrates_100g", 0.0),
12    "proteins_g": product_data.get("proteins_100g", 0.0),
13    "salt_g": product_data.get("salt_100g", 0.0),
14    "nutri_score": product_data.get("nutrition_score_fr_100g", "") or
        product_data.get("nutriscore_grade", ""),
15    "ecoscore_grade": product_data.get("ecoscore_grade", ""),
16    "labels": ", ".join(product_data.get("labels_tags", [])) if product_data.
        get("labels_tags") else "",
17    "off_json_data": product_data,
18 }

```

Das Mapping zeigt, dass die Integration nicht auf den Barcode reduziert ist. Vielmehr werden mehrere API-Felder auf Odoo-Felder abgebildet, darunter Nährwerte, Labels und Textinformationen. Die Rohdaten bleiben zusätzlich als JSON erhalten, was Debugging und spätere Erweiterungen erleichtert.

6.4 API-Aufruf, Validierung und Fehlerbehandlung

Der API-Zugriff ist zweistufig abgesichert. Zunächst wird der Barcode normalisiert, indem ein zwölfstelliger EAN-12-Code bei Bedarf mit führender Null ergänzt und ein dreizehnstelliger Code mit führender Null alternativ gekürzt wird. Anschließend werden zwei API-Versionen probiert, nämlich /api/v2/ und /api/v0/. Zusätzlich setzt der Code einen expliziten User-Agent und einen JSON-Accept-Header, weil open-food-facts generische Clients ablehnen kann.

Quellcode 7: Robuster Abruf von Open-Food-Facts-Daten

```

1 def _fetch_off_product(self, barcode):
2     headers = {
3         "User-Agent": "ERP-BarcodeOFF/1.0 (local-dev)",

```

```

4     "Accept": "application/json",
5 }
6 barcode_candidates = [barcode]
7 if barcode.isdigit() and len(barcode) == 12:
8     barcode_candidates.append(f"0{barcode}")
9 if barcode.isdigit() and len(barcode) == 13 and barcode.startswith("0"):
10     barcode_candidates.append(barcode[1:])
11
12 for candidate in list(dict.fromkeys(barcode_candidates)):
13     urls = [
14         f"https://world.openfoodfacts.org/api/v2/product/{candidate}.json",
15         f"https://world.openfoodfacts.org/api/v0/product/{candidate}.json",
16     ]
17     for url in urls:
18         response = requests.get(url, headers=headers, timeout=12)
19         if response.status_code == 404:
20             continue
21         response.raise_for_status()
22         payload = response.json()
23         if payload.get("status") == 1 and payload.get("product"):
24             return payload["product"]

```

Fehler werden als `UserError` an Odoo zurückgegeben. Das ist fachlich sinnvoll, weil der Benutzer im UI direkt eine verständliche Meldung erhält und die Transaktion abgebrochen wird. Damit verhindert die Implementierung inkonsistente Zwischenstände bei API-Ausfällen, HTTP-Fehlern oder fehlenden Produkten.

Quellcode 8: Validierung, Fehlermeldungen und Mengenbuchung

```

1 def action_import_from_open_food_facts(self):
2     self.ensure_one()
3     barcode = (self.barcode or "").strip()
4     if not barcode:
5         raise UserError(_("Please provide a barcode."))
6
7     product_data = self._fetch_off_product(barcode)
8     product = self._upsert_product(barcode, product_data)
9     self._apply_quantity(product)
10    self.product_tmpl_id = product.product_tmpl_id
11
12 def _apply_quantity(self, product):

```

```
13     if not self.quantity:
14         return
15     if not self.location_id:
16         raise UserError(_("Please select a stock location to update quantity."
17                             ))
18
19     self.env["stock.quant"].sudo()._update_available_quantity(product, self.
20         location_id, self.quantity)
```

Zusätzlich prüft die Kategorieermittlung die ersten Open-Food-Facts-Tags und legt bei Bedarf eine neue Produktkategorie an. Auch das ist ein Validierungsschritt, weil fehlende Kategorien nicht zu einem Abbruch führen, sondern kontrolliert ergänzt werden.

6.5 Transaktionslogik und Konsistenz

Die gesamte Importaktion läuft innerhalb der Odoo-Transaktion. Es gibt keine explizite manuelle Commit-Logik, keine Savepoints und keine direkte SQL-Verarbeitung. Wenn eine `UserError` oder eine `RequestException` auftritt, wird die Aktion abgebrochen und die Odoo-Transaktion nicht erfolgreich abgeschlossen. Dadurch bleiben Produkthanlage, Zusatzdaten und Mengenänderung konsistent.

Das ist wichtig, weil die Methode mehrere fachliche Schritte kombiniert: Barcode-Prüfung, API-Abfrage, Produkt-Upsert, Persistenz der Zusatzdaten und optionale Bestandsbuchung. Ein Fehler an beliebiger Stelle verhindert so, dass nur ein Teil der Daten geschrieben wird.

7 Fazit und Ausblick

Die Arbeit zeigt, dass eine automatisierte Stammdatenpflege im ERP-System Odoo durch Barcode-Erfassung und API-Integration technisch umsetzbar ist. Der entwickelte Prototyp verbindet die Eingabe oder den Scan eines Barcodes mit einer externen Produktdatenabfrage über die Open-Food-Facts-API und der anschließenden Erstellung oder Aktualisierung eines Produktdatensatzes in Odoo. Dadurch wird ein durchgängiger Prozess von der Produktidentifikation bis zur ERP-internen Weiterverarbeitung abgebildet.

Es wurde dargestellt, dass die Lösung insbesondere für die Reduktion manueller Erfassungsschritte geeignet ist, sofern die benötigten Produktdaten in der externen Datenquelle vorhanden und ausreichend vollständig sind. Die Daten können nach der Übernahme in Odoo für Lagerprozesse im Inventory-Modul und für nachgelagerte Verkaufsprozesse im Sales-Modul genutzt werden. Gleichzeitig wurde deutlich, dass die Qualität der automatisierten Produkthanlage von der Verfügbarkeit und Vollständigkeit der Open-Food-Facts-Daten abhängt.

Für zukünftige Arbeiten ergeben sich mehrere Erweiterungsmöglichkeiten. Zum einen könnten zusätzliche Produktdatenquellen integriert werden, um die Abhängigkeit von einer einzelnen externen Datenbank zu reduzieren. Zum anderen könnten erweiterte Validierungsregeln ergänzt werden, um unvollständige oder widersprüchliche API-Antworten systematischer zu behandeln. Darüber hinaus wäre eine mobile-first-orientierte Umsetzung denkbar, bei der Barcode-Erfassung, Produktdatenabfrage und Lagerbuchung stärker auf mobile Endgeräte ausgerichtet werden. Auch die Ergänzung bildbasierter Analysen könnte geprüft werden, sofern Produktbilder oder Verpackungsinformationen automatisiert ausgewertet werden sollen.

Codeverzeichnis

1	API-Endpunkt für Produktabfragen über Barcode	4
2	Produktbezogene API-Abfrage	14
3	Auszug aus <code>__manifest__.py</code>	17
4	Persistentes Modell für Open-Food-Facts-Daten	18
5	Dublettenprüfung und Upsert über den ORM	19
6	Mapping der API-Felder auf Odoo-Felder	20
7	Robuster Abruf von Open-Food-Facts-Daten	20
8	Validierung, Fehlermeldungen und Mengenbuchung	21

Literaturverzeichnis

- Chang, V. (2014). „Business Integration as a Service: Computational Risk Analysis for SMEs Adopting SAP“. In: *International Journal of Information Management* 34.3, S. 403–413.
- Chopra, S. und P. Meindl (2016). *Supply Chain Management: Strategy, Planning, and Operation*. Pearson.
- Davenport, T. H. (1998). „Putting the Enterprise into the Enterprise System“. In: *Harvard Business Review* 76.4, S. 121–131.
- Gronau, N. (2017). *Enterprise Resource Planning: Architektur, Funktionen und Management von ERP-Systemen*. München: De Gruyter Oldenbourg.
- ISO/IEC 15420 (2026). *ISO/IEC 15420:2025*. ISO. URL: <https://www.iso.org/standard/84892.html> (besucht am 07.07.2026).
- Krcmar, H. (2022). *Informationsmanagement*. 7. Aufl. Berlin: Springer Gabler.
- Lausen, H. (2020). *Datenqualität und Stammdatenmanagement*. Wiesbaden: Springer Gabler.
- Niedermeier, A. (2026). *Einführung in ERP-Systeme* | Springer Nature Link. URL: https://link.springer.com/chapter/10.1007/978-3-658-47758-5_2 (besucht am 07.07.2026).
- Odoo S.A. (2026a). *Inventory – Odoo 19.0 documentation*. Zugriff am 23.06.2026. URL: https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/inventory.html.
- (2026b). *ORM API – Odoo 19.0 documentation*. Zugriff am 23.06.2026. URL: <https://www.odoo.com/documentation/19.0/developer/reference/backend/orm.html>.
- (2026c). *Product management – Odoo 19.0 documentation*. Zugriff am 23.06.2026. URL: https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/inventory/product_management.html.
- (2026d). *Odoo Documentation — Odoo 19.0 Documentation*. URL: <https://www.odoo.com/documentation/19.0/index.html> (besucht am 07.07.2026).
- Open Food Facts (2026a). *Introduction to Open Food Facts API Documentation*. Offizielle API-Dokumentation. URL: <https://openfoodfacts.github.io/openfoodfacts-server/api/> (besucht am 26.06.2026).
- (2026b). *Tutorial on using the Open Food Facts API*. Offizielle Dokumentation zur Barcode-basierten Produktabfrage. URL: <https://openfoodfacts.github.io/openfoodfacts-server/api/tutorial-off-api/> (besucht am 26.06.2026).
- Romney, M. B. und P. J. Steinbart (2018). „Accounting Information Systems“. In: *Pearson Education*.

KI-Hilfsmittelverzeichnis

Nr.	KI-System	Einsatzzweck
1	ChatGPT	Formuliere einen wissenschaftlichen Einstieg für die Einleitung einer Seminararbeit zur End-to-End-Automatisierung von Produkt- und Lagerprozessen in Odoo. Berücksichtige ERP-Systeme, Produktstammdaten, Digitalisierung und Prozessqualität.
2	ChatGPT	Beschreibe die Problemstellung der manuellen Produktstammdatenpflege im Lebensmitteleinzelhandel. Gehe auf Fehleranfälligkeit, fehlende Standardisierung, redundante Datenerfassung und die Rolle von Barcodes ein.
3	ChatGPT	Formuliere eine Zielsetzung für eine Seminararbeit, in der Barcode-Erfassung, Open-Food-Facts-API und Odoo-Prozesse zu einer prototypischen End-to-End-Architektur verbunden werden.
4	ChatGPT	Schreibe einen kurzen Grundlagenabschnitt, der ERP-Systeme, Produktstammdaten, Barcodes und API-Schnittstellen als theoretische Basis der späteren Implementierung einordnet.
5	ChatGPT	Erstelle einen wissenschaftlich formulierten Abschnitt zur Anforderungsanalyse eines Odoo-Submoduls, das Barcodes verarbeitet, externe Produktdaten abrufen und Produkte automatisiert anlegt oder aktualisiert.
6	ChatGPT	Beschreibe Odoo als technische Systembasis für einen Prototypen mit Produktverwaltung, Inventory-Modul, Sales-Modul und projektspezifischer Erweiterung durch ein Custom-Addon.
7	ChatGPT	Formuliere den Einstieg in das Kapitel zur Konzeption und Umsetzung des Barcode-API-Submoduls. Der Fokus soll auf dem Prozess vom Barcode-Scan bis zum Produktdatensatz liegen.
8	ChatGPT	Formuliere eine technische Einordnung des Custom-Submoduls als Odoo-Addon mit Manifest, Modellen, Views und Sicherheitsdatei. Stelle die Schichten klar voneinander getrennt dar.
9	GitHub Copilot	Unterstütze bei der Strukturierung einer <code>__manifest__.py</code> für ein Odoo-Addon zur Open-Food-Facts-Integration. Berücksichtige Abhängigkeiten zu <code>product</code> , <code>stock</code> und <code>web</code> sowie die Einbindung von Security- und View-Dateien.
10	GitHub Copilot	Erzeuge einen Odoo-Modellcode für ein persistentes Modell <code>barcode.off.product</code> , das Barcode, Marke, Menge, Bild-URL, Zutaten, Allergene, Nährwerte, Scores, Labels und Rohdaten der API speichern kann.
11	GitHub Copilot	Entwickle einen Python-Codeausschnitt für eine Odoo-Upsert-Logik, die anhand eines Barcodes ein bestehendes Produkt sucht, vorhandene Produktvorlagen aktualisiert oder andernfalls ein neues Produkt erzeugt.
12	GitHub Copilot	Unterstütze bei der Erstellung eines Mapping-Dictionaries, das relevante Felder aus der Open-Food-Facts-API auf Odoo-Felder überträgt und zusätzlich die vollständige JSON-Antwort speichert.